

Numpy

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

¿Qué es numpy?

- numpy es una librería de Python para cálculo numérico. No viene incluida con Python; se instala aparte.
- Es la base de casi todo el ecosistema científico de python: pandas, xarray, matplotlib y scipy usan arreglos de numpy por dentro.
- numpy ofrece dos cosas:
 - Un nuevo tipo de objeto: **ndarray** (arreglo de n dimensiones)
 - Colección de **funciones** matemáticas que operan sobre él.

Ejemplo: sumar dos listas

- ¿Cómo sumamos las listas elemento a elemento? sin numpy!

```
a = [1, 2, 3, 4]  
b = [10, 20, 30, 40]
```


Ejemplo: sumar dos listas

- ¿Cómo sumamos las listas elemento a elemento? sin numpy!

```
a = [1, 2, 3, 4]
b = [10, 20, 30, 40]
```

- ¿Cómo sumamos las listas elemento a elemento con numpy?

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print(a + b)
```

Los operadores operan
elemento a elemento.

Probar: *, -

Vectorización

- **Vectorizar** es escribir una operación sobre el arreglo completo en lugar de un loop explícito sobre sus elementos.
- $a + b$ es una operación vectorizada.
- El loop no desaparece: numpy lo ejecuta por dentro, en código compilado (C), no en el intérprete de Python.
- En el loop de python, en cada vuelta el intérprete tiene que leer la instrucción, revisar que tipo tiene cada elemento, buscar como se suman los elementos, todo eso se repite muchas veces.
- numpy revisa el tipo una vez y después recorre los números de corrido.
- Idea importante: en numpy se piensa en **operaciones sobre arreglos completos**, no sobre elementos.

El ndarray

- numpy introduce un nuevo tipo de objeto: `ndarray`
- Parecido a las listas y tuplas: guarda una **colección de elementos** y se **indexa** con **corchetes**.
- Importante: **todos los elementos son del mismo tipo**.
- ¿Qué pasa si mezclamos diferentes tipos de objetos?

```
print(np.array([1, 2, 3.5]))  
print(np.array([1, "hola"]))
```


El ndarray

- El ndarray tiene tamaño fijo.

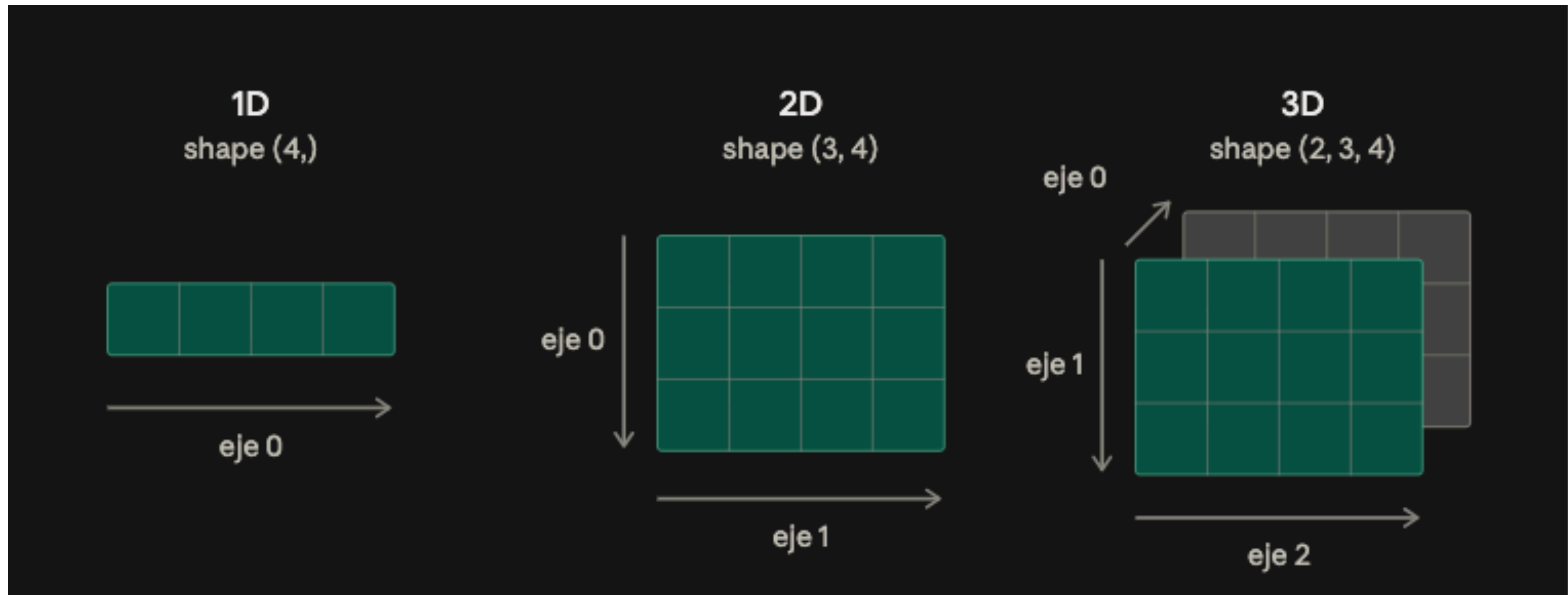
Dimensiones

- Un arreglo puede tener una, dos tres o más dimensiones.
- Un arreglo 1D es una fila de valores.
- Un arreglo 2D es una tabla.
- Un arreglo 3D son varias tablas apiladas.

Dimensiones

- Un eje (**axis**) es cada una de las direcciones en las que puedes moverte dentro del arreglo. Se numeran desde 0, en el mismo orden que se escriben los índices:
- En **a[i,j,k]**, i se mueve sobre el eje 0, j sobre el eje 1 y k sobre el eje 2.
- Reglas:
 - El eje 0 siempre es el primer índice, sin importar cuántas dimensiones tenga el arreglo.
 - En 2D es el de las filas (moverse hacia abajo), en 3D es el que elige la tabla.
 - **El último eje siempre es el de las columnas** (moverse a la derecha)
 - **shape** dice cuántos elementos hay a lo largo de cada eje. Ejemplo: (2, 3, 4) son 2 tablas de 3 filas por 4 columnas.

Dimensiones



Atributos

En ndarray tiene varios atributos, pero los importantes son:

- `ndim`: número de dimensiones
- `shape`: elementos a lo largo de cada eje
- `size`: total de elementos
- `dtype`: tipo de los elementos

Cómo se crean

- Desde listas:

```
a = np.array([1, 2, 3])  
b = np.array([[1, 2, 3], [4, 5, 6]])
```

¿qué dimensión y que shape tienen?

- ndim: número de dimensiones
- shape: elementos a lo largo de cada eje
- size: total de elementos
- dtype: tipo de los elementos

Cómo se crean

- Desde listas:

```
a = np.array([1, 2, 3])  
b = np.array([[1, 2, 3], [4, 5, 6]])
```

¿qué dimensión y que shape tienen?

Revisara todos los atributos

- ndim: número de dimensiones
- shape: elementos a lo largo de cada eje
- size: total de elementos
- dtype: tipo de los elementos

Cómo se crean

- Desde listas:

```
a = np.array([1, 2, 3])  
b = np.array([[1, 2, 3], [4, 5, 6]])
```

¿qué dimensión y que shape tienen?

Revisara todos los atributos

- Con valores iniciales, pasando el shape como tupla:

```
np.zeros((3, 4))      # todo ceros  
np.ones((3, 4))       # todo unos  
np.full((3, 4), 7.5)  # todo con el valor  
| | | | |           # que se indique  
np.empty((3, 4))      # ???
```


Cómo se crean

- Otros ejemplos:

```
print(np.array([(1, 2), (3, 4)]))  
print(np.array([(1, 2), (3, 4)]))
```

Cómo se crean

- Secuencias:

```
# como range, excluye el final  
np.arange(0, 10, 2)  
  
# n puntos, incluye el final  
np.linspace(0, 1, 5)
```

* usar arange para enteros y linspace para flotantes. Con pasos flotantes, arange puede dar un elemento de más o de menos por el redondeo.

Indexado y rebanadas

- ¿Qué nos devuelve el siguiente código?

```
t = np.array([10, 11, 12, 13, 14, 15])  
  
t[1:4]  
t[::2]  
t[1::2]  
t[::-1]
```

Indexado y rebanadas

- Funciona como listas, pero con un índice por eje, separados por coma.

```
a = np.arange(12).reshape(3, 4)
print(a)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

- ¿Qué nos devuelve lo siguiente?

```
a[1, 2]
a[0, :]
a[:, 1]
a[0:2, 1:3]
a[-1, -1]
```

Indexado y rebanadas

- Funciona como listas, pero con un índice por eje, separados por coma.

```
a = np.arange(12).reshape(3, 4)
print(a)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

- ¿Qué nos devuelve lo siguiente?

```
a[1, 2]
a[0, :]
a[:, 1]
a[0:2, 1:3]
a[-1, -1]
```

*nota: se puede escribir `a[1][2]` pero `a[1,2]` es la forma correcta en numpy.

Ejercicio

```
a = np.arange(72).reshape(3, 4, 6)
print(a)

print(a[1, 2])
print(a[0, :])
print(a[:, 1])
print(a[0:2, 1:3])
print(a[-1, -1])
```

- ¿Qué nos devuelven los prints?
- ¿Cómo accedo al elemento con el número 56?

Vistas

- Diferencia importante respecto a las listas.
- En una lista, rebanar crea una copia.
- En numpy, rebanar crea una **vista (view)**: un arreglo que hace referencia a los datos en el array original. El array original puede ser modificado usando la vista.

Vistas

- Diferencia importante respecto a las listas.
- En una lista, rebanar crea una copia.
- En numpy, rebanar crea una **vista**: un arreglo que hace referencia a los datos en el array original. El array original puede ser modificado usando la vista.
- ¿Cuál es la diferencia entre los dos bloques de código?

```
lista = [0, 1, 2, 3, 4]
sub = lista[1:3]
sub[0] = 99
print(lista)

arr = np.array([0, 1, 2, 3, 4])
sub = arr[1:3]
sub[0] = 99
print(arr)
```


Vistas

- Diferencia importante respecto a las listas.
- En una lista, rebanar crea una copia.
- En numpy, rebanar crea una vista: un arreglo que hace referencia a los datos en el array original. El array original puede ser modificado usando la vista.
- numpy usa vistas por eficiencia. Con datos grandes, copiar cada vez que rebanas sería muy caro. Si necesitas un arreglo independiente, se pide explícitamente.

```
sub = arr[1:3].copy()
```

```
lista = [0, 1, 2, 3, 4]
sub = lista[1:3]
sub[0] = 99
print(lista)

arr = np.array([0, 1, 2, 3, 4])
sub = arr[1:3]
sub[0] = 99
print(arr)
```

Indexado avanzado

- Con slicing se pueden sacar regiones ‘rectangulares’ y regulares: un rango de filas, un rango de columnas, cada n elementos.
- Muchas veces se necesita otra cosa: los elementos que cumplen una condición. Para esto numpy tienen dos herramientas: máscaras booleanas y fancy indexing.
- A diferencia del slicing, el indexado avanzado devuelve copias, no vistas.

Máscaras boleadas

- Una comparación sobre un arreglo es una operación vectorizada más: compara elemento a elemento y devuelve un arreglo de True/False del mismo shape. A ese arreglo se le llama máscara.

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
  
print(t > 22)
```

Máscaras booleanas

- Una comparación sobre un arreglo es una operación vectorizada más: compara elemento a elemento y devuelve un arreglo de True/False del mismo shape. A ese arreglo se le llama máscara.

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
  
print(t > 22)
```

- ¿Qué pasa si usamos la máscara como índice?

```
print(t[t>22])
```

Máscaras booleanas

- Una comparación sobre un arreglo es una operación vectorizada más: compara elemento a elemento y devuelve un arreglo de True/False del mismo shape. A ese arreglo se le llama máscara.

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
  
print(t > 22)
```

- ¿Qué pasa si usamos la máscara como índice?

```
print(t[t>22])
```

*se puede leer como 'los valores de t donde t es mayor que 22'

- En operaciones numéricas, True vale 1 y False vale 0.
- ¿Qué nos da el siguiente código?

```
print((t>22).sum())  
print((t>22).mean())
```

Condiciones

- Podemos usar &, | y ~ (no), con paréntesis en cada condición/

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])

print(t[(t > 20) & (t < 26)])
print(t[(t < 19) | (t > 27)])
print(t[~(t > 22)])
```

Arreglos de n dimensiones

- ¿Cuál es el resultado del siguiente código?

```
a = np.arange(12).reshape(3, 4)
print(a[a % 2 == 0])
```


Arreglos de n dimensiones

- ¿Cuál es el resultado del siguiente código?

```
a = np.arange(12).reshape(3, 4)
print(a[a % 2 == 0])
```

Numpy siempre devuelve el resultado en 1D

Máscara sobre un solo eje

```
a = np.array([[ 0,  1,  2,  3],
              [10, 11, 12, 13],
              [20, 21, 22, 23]])

print(a > 15)

# usamos la máscara como índice
print(a[a > 11])
print(a[[True, False, True], :])
print(a[:, [True, False, False, True]] )
```

- Máscara sobre un solo eje: es una lista de True/False con un valor por cada posición de ese *eje*, y va en el lugar de ese eje, igual que una lista de posiciones.

Asignar con máscaras

- Si la máscara va del lado izquierdo del =, sí se modifica el arreglo original.
- ¿Qué hace el siguiente código?

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
  
t[t < 20] = 20  
print(t)
```

Fancy indexing

- Es indexar con una lista (o arreglo) de posiciones: le dices exactamente qué posiciones quieres.
- Revisar que hace lo siguiente:

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
  
idx = [0, 2, 4]  
t[idx]  
t[[0, 2, 4]]
```

Fancy indexing

- Es indexar con una lista (o arreglo) de posiciones: le dices exactamente qué posiciones quieres.
- Revisar que hace lo siguiente:

```
print(t[[0, 2, 4]])  
print(t[[4, 0]])  
print(t[[1, 1, 1]] )
```

* puedes reordenar y repetir. Con slicing no puedes

Fancy indexing

```
a = np.array([[ 0,  1,  2,  3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
print(a)  
print(a[0,1])
```

renglón

columna

Fancy indexing

```
a = np.array([[ 0,  1,  2,  3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
print(a)  
print(a[0,1])  
print(a[[0,2], 3])
```

Se pone una lista donde antes
iba un número o slice

La lista nos dice que posiciones
queremos de ese eje

Fancy indexing

```
a[[0, 2], :]  
a[:, [0, 3]]  
  
a[[2, 0], :]  
a[:, [0, 1, 3]]  
a[[1, 1], :]
```


Fancy indexing

```
a[[0, 2], :]  
a[:, [0, 3]]  
  
a[[2, 0], :]  
a[:, [0, 1, 3]]  
a[[1, 1], :]
```

- * slicing: un slice solo describe posiciones con paso fijo.
- * Con una se puede pedir cualquier combinación, en cualquier orden repitiendo.

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.arange(12).reshape(3, 4)
print(a)
print(a[0,2])
print(a[[0, 2]])
print(a[:, [1, 3]])
```


Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0,  1,  2,  3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
filas = [0, 2]  
cols  = [1, 3]  
print(a[filas, cols])
```

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0,  1,  2,  3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
filas = [0, 2]  
cols  = [1, 3]  
print(a[filas, cols])
```

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0, 1, 2, 3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
filas = [0, 2]  
cols = [1, 3]  
print(a[filas, cols])
```

* No hace esto!

* Se leen como coordenadas

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0, 1, 2, 3],  
             [10, 11, 12, 13],  
             [20, 21, 22, 23]])  
  
filas = [0, 2]  
cols = [1, 3]  
print(a[filas, cols])
```

* No hace esto!

* Se leen como coordenadas

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0, 1, 2, 3],  
              [10, 11, 12, 13],  
              [20, 21, 22, 23]])  
  
filas = [0, 2]  
cols = [1, 3]  
print(a[filas, cols])
```

* No hace esto!

* Se leen como coordenadas

Fancy indexing

- ¿Qué hace el siguiente código?

```
a = np.array([[ 0, 1, 2, 3],  
             [10, 11, 12, 13],  
             [20, 21, 22, 23]])
```

```
filas = [0, 2]  
cols  = [1, 3]  
print(a[filas, cols])
```

```
print(a[[0, 2], :][:, [1, 3]])
```

- * No hace esto!
- * Se leen como coordenadas

Operaciones y reducciones

Funciones elemento a elemento

- numpy tiene versiones vectorizadas de las funciones matemáticas: se aplican a cada elemento y devuelven un arreglo del mismo shape.

```
x = np.array([1.0, 4.0, 9.0])

print(np.sqrt(x))
print(np.exp(x))
print(np.log(x))
print(np.abs(x))
print(np.round(x, 1))
print(np.sin(x)) # en radianes
```

Operaciones y reducciones

Reducciones

- Una reducción toma muchos valores y devuelve menos: típicamente uno solo.

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])

t.sum()      # suma
t.mean()     # promedio
t.min()      # mínimo
t.max()      # máximo
t.std()      # desviación estándar
```

* Se pueden escribir como método (`t.mean()`) o como función (`np.mean(t)`); es lo mismo.

Operaciones y reducciones

Reducciones con axis

- En más de una dimensión, sin axis la reducción usa todos los elementos y da un número. Con axis, reduce solo a lo largo de ese eje.

```
a = np.array([[1, 2, 3],  
              [4, 5, 6]])  
  
print(a.sum()) # devuelve un solo número  
print(a.sum(axis=0)) #????
```

Operaciones y reducciones

Reducciones con axis

- En más de una dimensión, sin axis la reducción usa todos los elementos y da un número. Con axis, reduce solo a lo largo de ese eje.

```
a = np.array([[1, 2, 3],  
              [4, 5, 6]])  
  
print(a.sum()) # devuelve un solo número  
print(a.sum(axis=0)) #????
```

* Pensar que el eje que indicas desaparece del shape.

* Ejemplo: si shape (2,3) con axis = 0 queda (3,)

Operaciones y reducciones

Reducciones con axis

- Ejemplo:

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
],
[
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
```

* Pensar que el eje que indicas desaparece del shape.

Operaciones y reducciones

Reducciones con axis

- Ejemplo:

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
],
[
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
```

- * Pensar que el eje que indicas desaparece del shape.
- * axis acepta una tupla para reducir varios ejes a la vez:
axis = (0,1)
- * keepdims=True deja el eje reducido con tamaño 1 en vez de quitarlo.

Operaciones y reducciones

Reducciones con axis

- Ejemplo:

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
],
[
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
```

```
print(a.shape)
print(a.sum(axis=(0)).shape)
```

```
(2, 3, 3)
(3, 3)
```

```
print(a.shape)
print(a.sum(axis=(0), keepdims=True).shape)
```

```
(2, 3, 3)
(1, 3, 3)
```

Posiciones

argmin, argmax, argsort

- A veces no queremos el valor máximo sino dónde esta.

```
b = np.array([1, 2, 3, 4, 10, 5])  
print(b.max())  
print(b.argmax())  
print(b[b.argmax()])
```


Posiciones

```
a = np.array([
    [
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ],
    [
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ]
])

print(a.max())
print(a.argmax()) # devuelve la posición como
                  # si el arreglo estuviera
                  # aplanado.
print(np.unravel_index(a.argmax(), a.shape))
```


Posiciones

```
a = np.array([
    [
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ],
    [
        [1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]
    ]
])

print(a.max())
print(a.argmax()) # devuelve la posición como
                  # si el arreglo estuviera
                  # aplanado.
print(np.unravel_index(a.argmax(), a.shape))
```

```
print(np.argwhere(a == a.max()))
```



```
a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
],
[
    [10, 2, 3],
    [4, 50, 6],
    [7, 8, 9]]
)

print(a.argmax(axis=1))
```

- * El shape (2,3,3) pierde el eje 1 y queda (2,3).
- * Para cada tabla y cada columna, dice en qué fila está el máximo.

```
[[2 2 2]
 [0 1 2]]
```

```

a = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
],
[
    [10, 2, 3],
    [4, 50, 6],
    [7, 8, 9]]
)

print(a.argmax(axis=1))

```

- * El shape (2,3,3) pierde el eje 1 y queda (2,3).
- * Para cada tabla y cada columna, dice en qué fila está el máximo.

```

[[2 2 2]
 [0 1 2]]

```

- * Todos los máximos están en la fila 2

sort y argsort

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
print(np.sort(t))  
print(np.argsort(t))
```


Acumulados

- cumsum: cada elemento es la suma de todos los anteriores.

```
t = np.array([18.2, 22.5, 25.1, 19.8, 27.3, 21.0])  
print(t.cumsum())
```

Broadcasting

Broadcasting es el mecanismo por el cual NumPy opera sobre arrays de shapes distintas.

En lugar de dar error, NumPy intenta "estirar" el array más pequeño para que encaje con el más grande.

Caso más simple:

```
a = np.array([[1,2,3]])  
print(a+10)
```

El 10 se repite para cada elemento. Pero NumPy no copia el número en memoria, solo lo trata como si lo hiciera.

Caso 2: array 1D sumado a uno 2D

```
m = np.array([[1, 2, 3], # shape (2, 3)  
              [4, 5, 6]])  
  
v = np.array([10, 20, 30]) # shape (3,)  
  
print(m + v)
```

NumPy estira v hacia abajo para cubrir las dos filas.

Broadcasting

Broadcasting es el mecanismo por el cual NumPy opera sobre arrays de shapes distintas.

En lugar de dar error, NumPy intenta "estirar" el array más pequeño para que encaje con el más grande.

Caso más simple:

```
a = np.array([[1,2,3]])  
print(a+10)
```

El 10 se repite para cada elemento. Pero NumPy no copia el número en memoria, solo lo trata como si lo hiciera.

Caso 2: array 1D sumado a uno 2D

```
m = np.array([[1, 2, 3], # shape (2, 3)  
              [4, 5, 6]])  
  
v = np.array([10, 20, 30]) # shape (3,)  
  
print(m + v)
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \underbrace{\begin{bmatrix} 10 & 20 & 30 \end{bmatrix}}_{\rightarrow \begin{bmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{bmatrix}} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{bmatrix}$$

NumPy estira v hacia abajo para cubrir las dos filas.

Broadcasting

Caso 3: sumar a cada columna

```
m = np.array([[1, 2, 3],      # shape (2, 3)
              [4, 5, 6]])

v = np.array([[10],          # shape (2, 1)
              [20]])

print(m + v)
```

Broadcasting

Caso 3: sumar a cada columna

```
m = np.array([[1, 2, 3],      # shape (2, 3)
              [4, 5, 6]])

v = np.array([[10],          # shape (2, 1)
              [20]])

print(m + v)
```

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} + \begin{bmatrix} 10 \\ 20 \end{bmatrix} \rightarrow \begin{bmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \end{bmatrix} = \begin{bmatrix} 11 & 12 & 13 \\ 24 & 25 & 26 \end{bmatrix}$$